

COMP3320/6464 Week 5, Lecture 3

Rhys Hawkins

Semester 2, 2024

Lab this week

- Writing code with Intel Intrinsics
- Examples in this lecture.

SIMD Data Types - Intel

Data Type	Content	SSE	SSE2	SSE3	AVX
__m64	8×char 4×short 2×int32 2×float 1×int64 1×double	✓	✗	✓	✓
__m128	4×float	✓	✓	✓	✓
__m128d	2×double	✗	✓	✓	✓
__m128i	16×char 8×short 4×int32 2×int64	✗	✓	✓	✓
__m256	8×float	✗	✗	✗	✓
__m256d	4×double	✗	✗	✗	✓
__m256i	32×char 16×short 8×int32 4×int64	✗	✗	✗	✓

Compiler Intrinsic Functions - Intel

SSE and AVX intrinsic function names use the following convention:
`mm_<opname>_<suffix>` `<opname>`: Indicates the basic operation of the intrinsic function E.g. `add` for addition and `sub` for subtraction `<suffix>`: Denotes the type of data the instruction operates on The first one or two letters of each suffix denote whether the data is:

- `p`: packed
- `ep`: extended packed
- `S`: scalar

The remaining letters and numbers denote the type, with notation as follows:

- `s`: single-precision floating-point
- `d`: double-precision floating-point
- `i32`: signed 32-bit integer
- `u32`: unsigned 32-bit integer
- etc

Compiler Intrinsic Functions - Examples

- Load four SP FP values, address aligned

- Intel SSE2: `_mm128 _mm_load_ps(float* p);`
- ARM NEON: `float32x4_t vld1q_f32 (const float32_t* p);`

R0	R1	R2	R3
p[0]	p[1]	p[2]	p[3]

- Store four SP FP values. Address must be 16-byte aligned

- Intel SSE2: `void _mm_store_ps(float* p, _mm128 a);`
- ARM NEON: `void vst1q_f32 (float32_t* p, float32x4_t a);`

p[0]	p[1]	p[2]	p[3]
a0	a1	a2	a3

Compiler Intrinsic Functions - Examples

- Add four SP FP values

- Intel SSE2: `_m128 _mm_add_ps(_m128 a, _m128 b);`
- ARM NEON: `float32x4_t vaddq_f32 (float32x4_t a, float32x4_t b);`

R0	R1	R2	R3
$a_0 + b_0$	$a_1 + b_1$	$a_2 + b_2$	$a_3 + b_3$

- Fused multiply-add four SP FP values

- Intel FMA3: `_m128 _mm_fmadd_ps(_m128 a, _m128 b, _m128 c);`
- ARM NEON: `void vfmmaq_f32(float32x4_t a, float32x4_t b, float32x4_t c);`

R0	R1	R2	R3
$a_0 * b_0 + c_0$	$a_1 * b_1 + c_1$	$a_2 * b_2 + c_2$	$a_3 * b_3 + c_3$

Compiler Intrinsic Functions - Portability

- Code vectorized with intrinsic functions for a specific processor is not portable
 - Only runs if architecture supports specific SIMD extension
- Portability ensured by using conditional compilation in code
 - Program must contain both a scalar and a vectorized version of the same computation
 - Using `#ifdef` statements, the applicable version is chosen at compile time
 - For e.g. in GCC, if `-msse2` option is passed, then the macro `SSE2` is defined
- Usage of conditional compilation:

```
#ifdef __SSE2__
    /* Code optimized with SSE2 intrinsic functions */
#elif __ARM_NEON__
    /* Code optimized with ARM NEON intrinsic functions */
#else
    /* Scalar code */
#endif
```

Optimizing SAXPY - scalar

- A simple case
- Vectorized well
- No loop carried dependency

```
void saxpy(int n, float a, float * __restrict__ X, float * __restrict__ Y)
{
    for (int i = 0; i < n; i++) {
        Y[i] = a*X[i] + Y[i];
    }
}
```


Optimizing SAXPY - loop unroll

- In many of my examples, I assume that the unroll amount is a factor of n .
- In general (i.e. for your assignment), you cannot assume this and need to compute any extra indices with a separate loop

```
void saxpy_x4(int n, float a, float * __restrict__ X, float * __restrict__ Y)
{
    int n4 = n - (n % 4);

    for (int i = 0; i < n4; i += 4) {
        Y[i] = a*X[i] + Y[i];
        Y[i + 1] = a*X[i + 1] + Y[i + 1];
        Y[i + 2] = a*X[i + 2] + Y[i + 2];
        Y[i + 3] = a*X[i + 3] + Y[i + 3];
    }

    for (int i = n4; i < n; i++) {
        Y[i] = a*X[i] + Y[i];
    }
}
```

Reference Manuals

Intel

- Intel Intrinsics Guide
- Intel 64 and IA-32 Architectures Software Developer's Manual

ARM

- ARM NEON Programmer's Guide (version 1.0)
- ARM NEON Intrinsics in GCC

Intel Intrinsic Guide

<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>

Intel® Intrinsic Guide
Updated 08/10/2022 Version 3.6.3

Instruction Set	Search Intel Archives
☐ MMX	
☐ SSE	
☐ SSE2	
☐ SSE3	
☐ SSE4P3	
☐ SSE4.1	
☐ SSE4.2	
☐ AVX	
☐ AVX2	
☐ FMA	
☐ AVX_VNNI	
☐ AVX_512	
☐ KNC	
☐ AMX	
☐ SVCL	
☐ Other	

Categories
☐ Application-Targeted
☐ Arithmetic

Intel Intrinsics Guide

Instruction Set

- ☐ MMX
- ☐ SSE
- ☐ SSE2
- ☐ SSE3
- ☐ SSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☐ AVX
- ☐ AVX2
- ☒ FMA
- ☐ AVX_VNNI
- ☐ AVX-512
- ☐ KNC
- ☐ AMX
- ☐ SVM
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Data Access
- ☐ Memory Access
- ☐ Other
- ☐ Streaming SIMD Extensions

Search Intel Intrinsics

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128 __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128 __m128d_ps (__m128d a, __m128d b, __m128d c)
```

Synopsis

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

```
__m128d __m128d_ps (__m128d a, __m128d b, __m128d c)
```

Description

Multiply packed single-precision (32-bit) floating-point elements in `a` and `b`, add the intermediate result to packed elements in `c`, and store the results in `dst`.

Operation

```
EXR [i] := 0 for 7  
i := 0  
for (i = 0; i < 4; i++)  
    dst[i*256:i+1] := (a[i*256:i+1] * b[i*256:i+1] + c[i*256:i+1])  
return  
dst[dst*256] := 0
```

Latency and Throughput

Architecture	Latency	Throughput (CP)
48-bit	4	0.5
Intel Core	4	0.5

Optimizing SAXPY - AVX

```
void saxpy_avx256(int n, float a, float *restrict X, float *restrict Y)
{
    int n8 = n - (n % 8);
    __m256 _a = _mm256_set1_ps(a);

    for (int i = 0; i < n8; i += 8) {
        __m256 xi = _mm256_loadu_ps(X + i);
        __m256 yi = _mm256_loadu_ps(Y + i);

        xi = _mm256_mul_ps(_a, xi);
        yi = _mm256_add_ps(xi, yi);

        _mm256_storeu_ps(Y + i, yi);
    }

    /* Second loop here */
}
```

- Note the intrinsics look like low level operations (load, mul, add, store)
- Recommend using unaligned version of load and store

Optimizing SAXPY - FMA

```
void saxpy_fma(int n, float a, float *restrict X, float *restrict Y)
{
    int n8 = n - (n % 8);
    __m256 _a = _mm256_set1_ps(a);

    for (int i = 0; i < n8; i += 8) {
        __m256 xi = _mm256_loadu_ps(X + i);
        __m256 yi = _mm256_loadu_ps(Y + i);

        yi = _mm256_fmadd_ps(_a, xi, yi);

        _mm256_storeu_ps(Y + i, yi);
    }
    /* Second loop here */
}
```

- FMA operation takes same amount of time as single add/mul operation

Optimizing Dot Product

Previously showed optimisation with loop unrolling and auxilliary variables and tested on Gadi.

- Dot product is a prime candidate for Fused Multiply Add (FMA) operations
- Also has a loop carried dependency (s)

```
float dot(int N, float *restrict A, float *restrict B) {  
    float s = 0.0;  
    for (int i = 0; i < N; i++) {  
        s = s + A[i] * B[i];  
    }  
    return s;  
}
```

Optimizing Dot Product : Loop peel and unroll

```
float dot_x4(int N, float *restrict A, float *restrict B) {
    float s1 = A[0] * B[0];
    float s2 = A[1] * B[1];
    float s3 = A[2] * B[2];
    float s4 = A[3] * B[3];
    int N4 = N - (N%4);
    for (int i = 4; i < N4; i += 4) {
        s1 = s1 + A[i] * B[i];
        s2 = s2 + A[i + 1] * B[i + 1];
        s3 = s3 + A[i + 2] * B[i + 2];
        s4 = s4 + A[i + 3] * B[i + 3];
    }

    for (int i = N4; i < N; i += 1) {
        s1 = s1 + A[i] * B[i];
    }

    return (s1 + s2) + (s3 + s4);
}
```

- Loop unrolling with auxiliary variables takes better advantage of pipelining
- Loop peeling saves a small amount but can be significant in inner loops

Optimizing Dot Product : FMA

```
#include <immintrin.h>

float dot_fma(int N, float *restrict A, float *restrict B) {

    __m256 ai = _mm256_loadu_ps(A + 0);
    __m256 bi = _mm256_loadu_ps(B + 0);

    __m256 s = _mm256_mul_ps(ai, bi);

    int N8 = N - (N%8);
    for (i = 8; i < N8; i += 8) {
        ai = _mm256_loadu_ps(A + i);
        bi = _mm256_loadu_ps(B + i);

        s = _mm256_fmadd_ps(ai, bi, s);
    }

    /* AVX Reduction */
}
```

Optimizing Dot Product : AVX Reduction

- We have 8 float values stored in `s` and we need to return their sum
- Use SSE instructions and registers.

```
/* Step 1: Add lower and upper halves to reduce 8 values down to 4 */
__m128 r4 = _mm_add_ps(_mm256_castps256_ps128(s), _mm256_extractf128_ps(s, 1));

/* Step 2: Add lower and upper 2 floats to reduce 4 values down to 2 */
__m128 r2 = _mm_add_ps(r4, _mm_movehl_ps(r4, r4));

/* Step 3: Add lower and upper floats to reduce 2 values down to 1 */
/* (note ss suffix) */
__m128 r1 = _mm_add_ss(r2, _mm_movehdup_ps(r2));

/* Convert to float output */
return _mm_cvtss_f32(r1);
```

Optimizing Dot Product : FMA/AVX + Loop unroll

- Modern Intel CPUs can initiate multiple SIMD instructions per cycle
- Compiler intrinsics + Loop unrolling is possible

```
#include <immintrin.h>
```

```
float dot_fma_x2(int N, float *restrict A, float *restrict B) {
```

```
    /* Preamble */
```

```
    for (i = 16; i < N16; i += 16) {  
        a1 = _mm256_loadu_ps(A + i);  
        a2 = _mm256_loadu_ps(A + i + 8);  
  
        b1 = _mm256_loadu_ps(B + i);  
        b2 = _mm256_loadu_ps(B + i + 8);  
  
        s1 = _mm256_fmadd_ps(a1, b1, s1);  
        s2 = _mm256_fmadd_ps(a2, b2, s2);  
    }
```

```
    _mm256 s = _mm256_add_ps(s1, s2);
```

```
    /* AVX Reduction */
```

```
}
```

Verification on GADI

- Count cycles and floating point operations using PAPI
- Testing with repeat resulting in approx. 20.5 Billion FLOPs

	G. Cycles	FLOPs/cycle	MFLOPs
Original	40.98	0.5	1800
Loop Unroll x2	20.49	1.0	3600
Loop Unroll x4	10.25	2.0	7300
AVX/FMA	4.10	5.0	18000
AVX/FMA Unroll x2	2.34	8.8	31000
AVX/FMA Unroll x4	1.51	13.6	48000

- Added benefit of FMA operations, improvements in accuracy
- Loop unroll x4 5.11×10^{-7} relative error
- AVX/FMA 0.74×10^{-7} relative error

Summary

- Compiler intrinsics are a relatively simple way to utilize SIMD instructions directly in code
- Benefits over auto-vectorization
 - Can be applied without restriction (i.e. where auto-vectorization may not be possible)
 - No need to use potentially dangerous compiler flags (`-ffast-math`) that would apply over whole code base
 - More deterministic behavior across platforms/compilers
- Be careful with un/aligned loads/stores
- Extra testing/verification required for different loop lengths (e.g. Assignment 1)